SKILL - 2

Name: Hansika Reddy Gunapati

Rollno: 2520030237

Section: 8

Aim

To create an interactive main loop that displays a prompt, reads user input, handles exit conditions, and manages keyboard input including Backspace and Enter keys for multi-character commands.

Also, to design a clear control flow for the interactive loop, manage an input buffer, support multi-character commands, and test the complete user interaction.

Objective

- To create a continuous main loop.
- To display a prompt and read user input.
- To handle exit conditions correctly.
- To capture keyboard input.
- To handle the Backspace key.
- To process the Enter key.
- To manage an input buffer.
- To support multi-character commands.
- To design and understand the control flow of the loop.
- To test the interactive user input process.

Problem Statement

Design an interactive command-line loop that continuously accepts keyboard input from the user. The program should display a prompt, collect characters into an input buffer, allow correction using Backspace, recognize Enter as the end of the command, process multi-character commands, and terminate when the exit condition is satisfied.

Theory

An interactive command-line program repeatedly communicates with the user. Instead of executing only once, the program waits for input, processes the entered command, and returns to the prompt for another command. This repeated behavior is implemented using a main loop.

1. Main Loop

The main loop is the central control structure of the program. It repeatedly displays the prompt, accepts input, processes the completed command, checks the exit condition, and starts another iteration when required.

2. Display Prompt

A prompt tells the user that the program is ready to receive input. After a normal command is processed, the prompt is displayed again so that interaction can continue.

3. User Input

Keyboard input consists of characters entered by the user. The program examines each character and determines whether it is a normal character, Backspace, or Enter.

4. Input Buffer

The input buffer temporarily stores the characters that make up the current command. Characters are added sequentially, and the buffer position is adjusted when Backspace is used.

5. Multi-Character Commands

A command such as hello is formed by storing several characters in sequence. The buffer preserves their order until Enter is pressed and the complete command is processed.

6. Keyboard Input Processing

6.1 Backspace Handling

When Backspace is pressed, the program checks whether the input buffer contains a character. If it does, the last character is removed by decreasing the buffer index. This allows the user to correct the current command.

6.2 Enter Key Handling

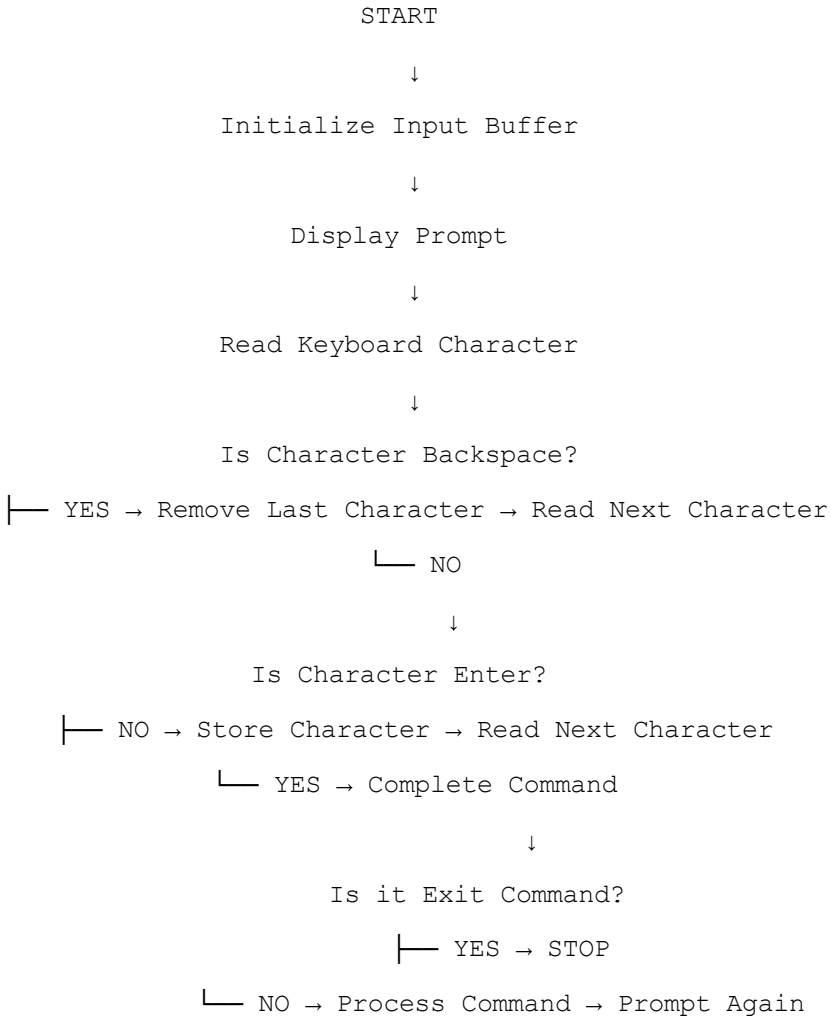
Enter indicates that the user has completed the current command. The program terminates the string in the buffer and processes the complete command.

6.3 Exit Condition

An exit condition provides a controlled way to stop the interactive loop. In this practical, the command exit is checked after Enter is pressed. If it matches, the program terminates.

6.4 Control Flow

The control flow of the interactive loop is represented below:



6.5 Explanation

The program starts by initializing the buffer and displaying the prompt. It then reads characters one by one. Backspace edits the buffer, normal characters are stored, and Enter completes the command. The completed command is checked for the exit condition before the next loop iteration begins.

7. Algorithm

1. Start the program.
2. Declare the input buffer, character variable, and buffer index.
3. Initialize the buffer index to zero.
4. Display the command prompt.
5. Read a keyboard character.
6. Check whether the character is Backspace.
7. If Backspace is pressed and the buffer is not empty, remove the last character.
8. If it is not Backspace, check whether it is Enter.
9. If it is not Enter, store the character in the input buffer.
10. Continue reading characters until Enter is pressed.
11. Terminate the input buffer with a string terminator.
12. Compare the completed command with the exit command.
13. If the command is exit, display the exit message and terminate.
14. Otherwise, process the command.
15. Reset the buffer and display the prompt again.
16. Repeat until the exit condition is satisfied.
17. Stop the program.

8. Data Used

A character array is used as the input buffer. An integer index identifies the current position in the buffer, while a character variable temporarily stores each keyboard character.

9. Operations Used

- Character input is read continuously.
- Normal characters are stored in order.
- Backspace removes the last stored character.
- Enter completes the command.
- String comparison checks the exit command.
- The main loop repeats for non-exit commands.

10. Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];
    char ch;
    int index;

    while (1) {
        index = 0;
        printf("\n> ");

        while ((ch = getchar()) != '\n') {
            if (ch == '\b') {
                if (index > 0) {
                    index--;
                    printf("\b \b");
                }
            } else if (index < 99) {
                input[index++] = ch;
                putchar(ch);
            }
        }

        input[index] = '\0';

        if (strcmp(input, "exit") == 0) {
            printf("\nExiting program...\n");
            break;
        }

        printf("\nCommand entered: %s\n", input);
    }

    return 0;
}
```

11. Program Explanation

11.1 Header Files

stdio.h provides standard input and output functions. string.h provides the strcmp() function used to compare the entered command with the exit command.

11.2 Input Buffer

The input array stores the characters entered by the user. The index variable keeps track of the next available position.

11.3 Main Loop

while(1) keeps the program running continuously. The break statement terminates the loop when the exit command is detected.

11.4 Character Processing

Each character is checked. Normal characters are stored, Backspace removes the last character when possible, and Enter completes the command.

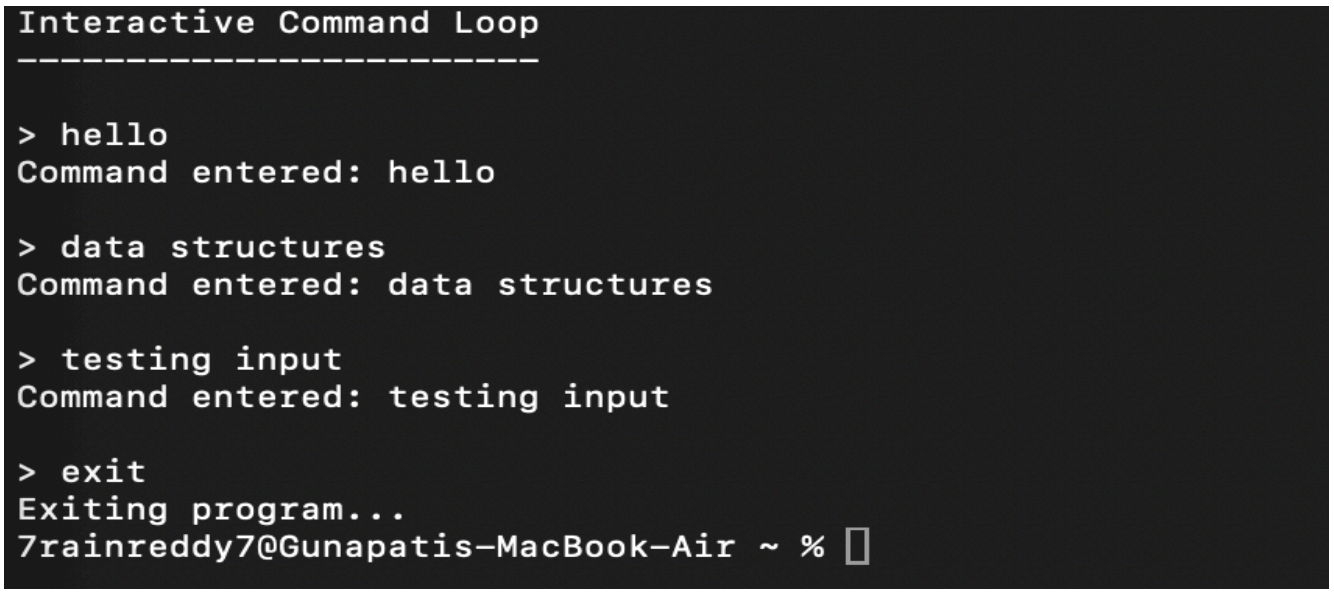
12. Sample Output

The following is an example of the expected program output:

```
> hello
Command entered: hello

> data
Command entered: data

> testing
Command entered: testing

> exit
Exiting program...
```

13. Output Screenshot

Paste the screenshot of the program output in the space provided below.



14. Testing

- Tested with a single-character command.
- Tested with multi-character commands.
- Tested Backspace correction.
- Tested Enter to complete the command.
- Tested the exit command.
- Verified that the prompt reappears after normal commands.

15. Test Cases

Test Case	Input	Expected Result
1	hello	Command is accepted and displayed.
2	data	Multi-character command is accepted.
3	test	Command is processed normally.
4	Backspace	Last entered character is removed.
5	Enter	Current command is completed.
6	exit	Program terminates successfully.

16. Observation

- The main loop continuously displayed the prompt.
- Keyboard characters were captured and stored in the input buffer.
- Multi-character commands were successfully supported.
- Backspace allowed previously entered characters to be removed.
- Enter completed the current command.
- The exit condition successfully stopped the loop.
- The prompt returned after processing normal commands.
- The testing process confirmed the expected control flow.

17. Result

The interactive main loop was successfully implemented and tested. The program displayed a prompt, captured keyboard input, maintained an input buffer, handled Backspace and Enter keys, supported multi-character commands, and correctly terminated when the exit condition was satisfied.

18. Conclusion

The practical demonstrated the basic structure of an interactive command-line program. The main loop, input buffer, keyboard processing, editing operations, command completion, and exit checking worked together to provide continuous user interaction.